

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/360009648>

Using Convolution Neural Networks to Predict the Performance of Footballers in the Fantasy Premier League

Technical Report · April 2022

DOI: 10.13140/RG.2.2.10010.72645/2

CITATION

1

READS

1,744

1 author:



Delano Ramdas

2 PUBLICATIONS 1 CITATION

SEE PROFILE

Using Convolution Neural Networks to Predict the Performance of Footballers in the Fantasy Premier League

Delano Ramdas

Student Number 60241500

Supervised by Professor Etienne van der Poel

Abstract

Successful performance in the Fantasy Premier League requires accurate forecasting of the footballers that are likely to perform well in the near future. This report investigates the application of Convolutional Neural Networks to this prediction problem. The results indicate that feeding convolutional neural networks with input data represented as a time-series, yields a predictor that outcompetes existing champion techniques in the machine learning domain, for the fantasy premier league prediction problem.

Keywords: Convolutional Neural Networks, Fantasy Premier League, Time Series Forecasting

Contents

1	Introduction	1
1.1	The Fantasy Premier League	1
1.2	Predicting FPL Footballer Performance	1
1.3	Research Direction	2
1.4	Methodology Overview	2
1.5	Limitations and Contributions	3
1.6	Report Structure	3
2	Literature Review	3
2.1	Related Work on Similar Problems	3
2.2	Related Work on the FPL Prediction Problem	4
2.3	Machine Learning Techniques for Time-Series Data	6
2.4	Neural Network Theory	6
2.4.1	Perceptrons	6
2.4.2	Multi-layer Perceptrons	7
2.4.3	Long Short-Term Memory Recurrent Neural Networks	8
2.4.4	Convolutional Neural Networks	8
2.5	Training and Evaluation	9
2.6	Datasets and pre-processing	10
3	Research Methodology	11
3.1	Data Pre-processing	11
3.2	Time-series Representation for Input Data	12
3.3	Network Architecture, Hyperparameters, Training and Evaluation	13
3.3.1	CNN	14
3.3.2	Networks used for Comparison	14
3.3.3	Training	15
3.3.4	Evaluation and Analysis	15
3.4	Programmatic Implementation	15
3.5	Ethical Considerations	15
4	Data Analysis and Results	15
4.1	Training the CNN	15
4.2	Optimal Configuration for CNN	16
4.3	Comparison with other networks	16
5	Conclusion	25

List of Figures

1	Graphic comparison of the prediction results of different machine learning techniques (Lindberg and Sonderberg 2020)	5
2	Model of a Perceptron (Nielsen 2015)	7
3	Model of an MLP (Mitchell 1997)	7
4	ReLU Function	8
5	Example architecture of LeNet (Hornik 1991)	9
6	Overview of end-to-end operations in a CNN leveraging 1D convolutions for time-series data (Granat 2019)	14
7	MSE over 100 epochs for goalkeepers	17
8	MSE over 100 epochs for defenders	17
9	MSE over 100 epochs for midfielders	18
10	MSE over 100 epochs for forwards	18
11	MSE per hyperparameter combination for goalkeepers with a time interval of 3 game-weeks	19
12	MSE per hyperparameter combination for goalkeepers with a time interval of 6 game-weeks	19
13	MSE per hyperparameter combination for goalkeepers with a time interval of 9 game-weeks	20
14	MSE per hyperparameter combination for defenders with a time interval of 3 gameweeks	20
15	MSE per hyperparameter combination for defenders with a time interval of 6 gameweeks	21
16	MSE per hyperparameter combination for defenders with a time interval of 9 gameweeks	21
17	MSE per hyperparameter combination for midfielders with a time interval of 3 game-weeks	22
18	MSE per hyperparameter combination for midfielders with a time interval of 6 game-weeks	22
19	MSE per hyperparameter combination for midfielders with a time interval of 9 game-weeks	23
20	MSE per hyperparameter combination for forwards with a time interval of 3 gameweeks	23
21	MSE per hyperparameter combination for forwards with a time interval of 6 gameweeks	24
22	MSE per hyperparameter combination for forwards with a time interval of 9 gameweeks	24

List of Tables

1	Extract of historic FPL statistics for Tottenham Hotspur player Heung-Min Son (<i>FPL Statistics for Heung-Min Son 2021</i>)	1
2	Extract of a player csv	11
3	Data Features	12
4	Optimal MLP Architecture (Lindberg and Sonderberg 2020)	14
5	Optimal LSTM RNN Architecture (Lindberg and Sonderberg 2020)	15
6	Optimal CNN configuration per position	16
7	Comparison of the MSE for different machine learning techniques applied to the FPL prediction problem (the lower, the better)	25

Acronyms

ARIMA Autoregressive Integrated Moving Average.

CNN Convolutional Neural Network.

EPL English Premier League.

FPL Fantasy Premier League.

LSTM RNN Long Short-Term Memory Recurrent Neural Network.

MLP Multi-Layer Perceptron.

MSE Mean Squared Error.

ReLU Rectified Linear Unit.

SVM Support Vector Machine.

1 Introduction

1.1 The Fantasy Premier League

‘Fantasy Football’ is a genre of online games in which game players (called ‘managers’) select line ups of footballers to construct virtual football teams. The Fantasy Premier League (FPL) is a global fantasy football game tied to the English Premier League (EPL). FPL is considered the largest fantasy football game in the world with a record of over 7 million managers during the 2020/2021 EPL season (*Fantasy Premier League 2021*).

Each selected footballer in a fantasy team accrues points based on the real-life performance of that football player during some football tournament. Each football player is awarded fantasy points based on a variety of factors including (but certainly not limited to) the number of goals they score, the position they play in and whether they maintain a clean sheet. For example, in FPL, a defender is awarded 4 points for maintaining a clean sheet and 6 points for scoring a goal. The complete, comprehensive set of rules, constraints and point-accrual criteria can be found at the official FPL website (*Fantasy Premier League Rules 2021*).

For every round of matches in the tournament (called ‘gameweeks’), managers attempt to predict which line up of footballers are likely to perform well during that gameweek. Managers often consult footballers’ statistics and expert predictions to inform their picks. The FPL ecosystem maintains an extensive dataset of footballers’ statistics with a variety of data features like the number of red cards that a player had been issued or the number of their successful passes that have led to a goal. This dataset can be interrogated for insight to help forecast the likely points haul for a player in an upcoming gameweek - thereby indicating whether a manager should field them or not. Table 1 displays a small subset of statistics available for a player.

Gameweek	assists	bonus	clean_sheets	...
1	1	11	6	...
2	9	13	10	...
3	8	16	14	...
4	8	17	10	...
5	13	18	7	...

Table 1: Extract of historic FPL statistics for Tottenham Hotspur player Heung-Min Son (*FPL Statistics for Heung-Min Son 2021*)

1.2 Predicting FPL Footballer Performance

It follows from above that there exists an opportunity to apply machine learning techniques to fantasy football datasets in order to predict footballers’ performance (how many fantasy points they will score) during the next gameweek. A more formal modelling of the problem follows.

Given the input space X , output space Y , function $f : X \rightarrow Y$ which maps the input space to the output space, and the hypothesis space H which contains hypothesis functions that could possibly satisfy f , is it possible to employ a machine learning technique to choose or incrementally refine hypothesis h from H such that $h \approx f$?

In the context of the fantasy premier league prediction problem, the input space X includes factors which contribute toward player performance, and the output space Y is a continuous variable representing the number of points a footballer accrues (which is indicative of his performance). Thus, this report will leverage a machine learning technique to choose hypothesis function h which approximates f i.e the function that outputs the fantasy points that a football player accrues in some gameweek, given a set of statistics about that footballer's performance in gameweeks prior.

A variety of machine learning techniques have been applied to fantasy football datasets with varying degrees of success. However, to the best of the authors knowledge, no previous attempt has been made to employ a Convolutional Neural Network (CNN); which has been shown to be especially useful upon time-series data representations. Chapter 2 elaborates on this material in detail.

1.3 Research Direction

This report aimed to use a convolutional neural network to predict the future performance of footballers in the fantasy premier league based on their performance metrics in gameweeks prior. In doing so, the following research question was addressed: 'How do Convolutional Neural Networks perform in the prediction of player points in the Fantasy Premier League?'

Here, performance is defined as the accuracy of the predictions on a test dataset. This was measured by the mean squared error loss function, where a lower loss meant better predictions and therefore better performance.

This research served to fulfil two objectives. Firstly, to successfully implement a CNN programmatically to predict the score of fantasy footballers in the fantasy premier league (should it be determined to be possible). Secondly, to compare the performance of a CNN to other machine-learning techniques that have proven successful at the same problem.

These objectives will answer two research sub-questions. Firstly, can a CNN be successfully implemented to predict player performance in FPL? Secondly, how does the performance of CNNs compare to a Long Short-Term Memory Recurrent Neural Network (LSTM RNN) and a Multi-Layer Perceptron (MLP), which were both previously shown to be powerful predictors on the FPL dataset, in predicting player points in FPL?

1.4 Methodology Overview

In order to meet these objectives the following process was followed.

First, significant data extraction and pre-processing was conducted on the FPL dataset.

Next, networks with different combinations of parameters were trained and tested in order to determine an optimal configuration for the CNN. The networks were trained by feeding in cleaned statistics of football players' past performance, represented as a time-series, in order to output a probable points return for that player in the following gameweek.

Thereafter, the network's mean squared error was measured on a test set, and compared against an LSTM RNN and an MLP.

Finally, the results were compared to literature, finding that CNNs can indeed predict the performance of footballers in the fantasy premier league; and can well outperform other machine learning techniques that had been previously reported to have performed best.

1.5 Limitations and Contributions

The most impactful limitation to this experiment is the availability of usable data. Fantasy football datasets are either difficult to access or stored in a format that requires significant pre-processing in order to be usable in a machine learning setting. Therefore, far less data than preferred was used for testing and training the models.

Automated solutions to the FPL prediction problem provide managers with an additional data point to consider when picking players. Accurate solutions yield value by providing a competitive advantage for FPL managers, which is especially important in peer-to-peer and competitive fantasy leagues. Gupta (2019) argues that making accurate predictions about fantasy teams can funnel into real-life sports analytics – as decision-making aids for football team managers. Gangal et al. (2015) presents an FPL predictor as a unique opportunity to incorporate a betting-esque ‘draft’ feature into the game – using the predictor to determine the bet odds.

This report extends the body of knowledge in the ‘applications of machine learning in fantasy football’ domain by showing that a previously unused technique, Convolutional Neural Networks, can be successfully applied to fantasy football datasets to predict the future performance of footballers and can yield a powerful predictor.

1.6 Report Structure

The balance of this report will be organised as follows. Chapter 2 presents insight from related research in the football and fantasy football domains, and explores neural network theory. Chapter 3 outlines the experiments’ methodology while chapter 4 details the outcomes of those experiments. Chapter 5 concludes the report with a summary and discussion on limitations and future research.

2 Literature Review

2.1 Related Work on Similar Problems

A variety of techniques have been applied to sport-related prediction problems over the years. As early as 1999, machine learning techniques had been applied to predicting the outcome of sport matches. Pardee (1999) used MLPs to predict the outcome of basketball matches in the National Football League. Sierra, Fosco, and Fierro (2020) and Hamadani (2005) did the same using Support Vector Machines (SVMs). In the English Premier League, variants of K Nearest Neighbours (Timmaraju, Palnitkar, and Khanna 2013) and Gradient Boosted Trees (Hubáček, Šourek, and Železný 2019) have been used to predict the winning team in head-to-head matches.

The mathematics and statistics disciplines were among the first to apply algorithms to predict player performance within fantasy sport games. In his blog, Rudy (2011) experimented with regression analysis over the NFL fantasy dataset. Matthews, Ramchurn, and Chalkiadakis (2012) formally applied Q-Learning to FPL, generating the first recorded FPL predictor. Mathematical Programming methods were applied a year later to GrandT, an Argentinian fantasy football tournament (Bonomo, Duran, and Marengo 2013).

The early literature proved that models from the statistics, mathematics and computer science domains could be successfully applied to real and fantasy sport datasets.

Lutz (2015) was perhaps the first to apply common machine learning techniques to predict player performance in a fantasy football setting. He trialed both an SVM and MLP, on a narrow dataset that contained only quarterback players.

Given that fantasy datasets often contain very many features, the choice of SVM was appropriate since SVMs can construct linear decision surfaces in high-dimension feature spaces (Cortes and Vapnik 1995). However, Lutz made a critical assumption that the decision surface would be linear.

MLPs typically leverage gradient descent over a network of perceptrons to express nonlinear decision surfaces (Mitchell 1997). Thus, the MLP was an appropriate contrast to the SVM, should the decision surface turn out to be nonlinear.

Both approaches yielded a high absolute error with little insight over the nature of the decision surfaces. However, Lutz’ work showed that machine learning techniques could be successfully applied to predicting player performance in fantasy football games.

2.2 Related Work on the FPL Prediction Problem

Stolyarov and Vasiliev (2017) is the first recorded attempt to use common machine learning techniques to predict player performance in FPL. The authors leveraged gradient boosted trees (specifically, XGBoost) to predict the performance of players. XGBoost is an enhanced greedy function approximator (T. Chen and Guestrin 2016) premised upon gradient boosting machines (Friedman 2001). A year later, Raghunandh (2018) did the same. Both Stolyarov and Vasiliev, and Raghunandh used their predictions in extended contexts without providing analysis over the accuracy of the predictions themselves.

Saifi (2018) used both gradient boosted machines and random forests to analyse prediction accuracy when additional, unofficial data features (expectedGoals and expectedAssists) are added to the FPL dataset. Similarly, Bonello et al. (2019) combined gradient boosted machines with expert predictions scraped from twitter and the general web. Both Saifi (2018) and Bonello et al. (2019) analysed the improvement in prediction accuracy with their additional data features without commenting on the accuracy of the techniques in their baseline form.

Time-series data refers to values that have been collected sequentially and recorded with awareness of their time dimension (Esling and Agon 2012). Gupta (2019) viewed FPL predictions as a time-series problem. As a result he combined Long Short-Term Memory Recurrent Neural Networks with an Autoregressive Integrated Moving Average (ARIMA) to predict player performance.

LSTMs are gradient-based recurrent neural networks that can hold long-term information via a cell state that is regulated by gates (Hochreiter and Schmidhuber 1997). LSTMs improve on regular RNNs by using the cell state to store important gradients over a long period of time, which would otherwise vanish (Olah 2015).

At its core, ARIMA is a regression model which measures the strength of variables in relation to each other.

This pairing leverages the power of LSTM RNNs in modeling non-linear time-series data (Gers 2001) with ARIMA’s ability to handle non-periodic data (Gupta 2019).

Gupta extended the model to select an optimal ‘dream team’ within the game’s rule-set, by leveraging the simplex method from linear programming. Consequently, Gupta’s results analysis evaluated the difference in predicted points accrued and actual points accrued for the entire team over an entire season. Gupta’s best solution slightly overfitted the data - underpredicting the cumulative team points total by 87 points. The model was unable to account for sudden player absences in the test set.

Lindberg and Sonderberg (2020) have provided the most comprehensive analysis of predicting player scores in FPL using machine learning algorithms, to date. Instead of working to output a ‘dream team’ constrained by game rules, the authors focused rather on exploring different machine learning techniques that can solve the FPL prediction problem - SVMs, MLPs and LSTM RNNs with different hyperparameters.

The problem was investigated two ways - as a classification problem and as a regression one. The classification models aimed to secure a ‘good result’ even if it was not optimal. The regression models pursued a maximal points haul.

As a classification problem, SVMs performed the best and time-series models, the worst. Notably, Ulmer and Fernandez (2014), in addition to Timmaraju, Palnitkar, and Khanna (2013) found that using SVM classifiers in a different domain - predicting the outcome of football matches - suffered the same problem of high entropy that Lindberg and Sonderberg (2020) did.

As a regression problem, the time-series models outperformed the other models across all player positions, measured using a mean squared error loss function. Somewhat expectedly, support vector regression performed the worst.

In both classification and regression framings, MLPs consistently placed in the middle. Crucially, all the regression models far outperformed their classification counterparts and all Lindberg and Sonderberg’s models outperformed the global average on the test set. Figure 1 present a summary of the results of Lindberg and Sonderberg’s thesis.

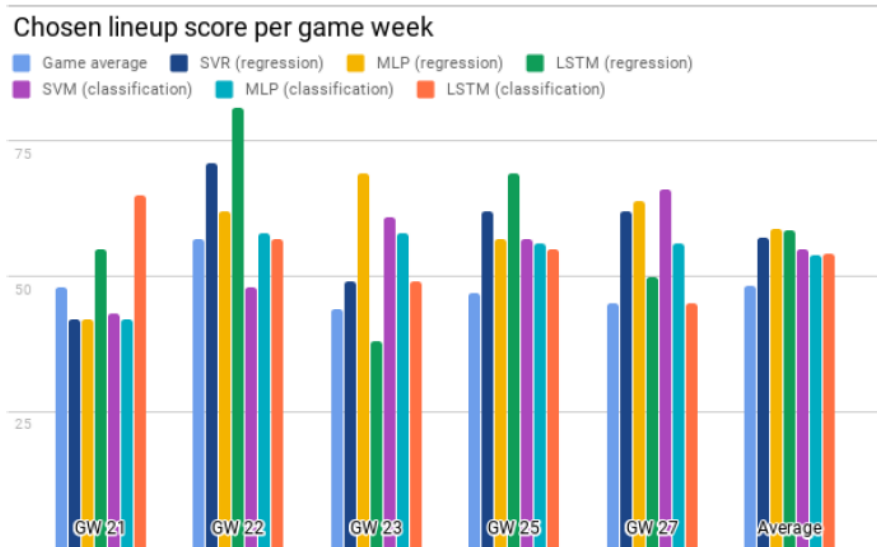


Figure 1: Graphic comparison of the prediction results of different machine learning techniques (Lindberg and Sonderberg 2020)

The neural networks outperformed the SVMs since they made no assumptions about the data or relationships between variables - unlike the linearity-bias inherent to SVMs. Neural networks have been well-suited to model complex, non-linear data across a variety of domains from ozone forecasting (Comrie 1997) and tornado formation (Marzban and Stumpf 1996) to stock markets (Thakkar and Chaudhari 2021) and now, fantasy football too.

The outstanding performance of the time-sensitive LSTM RNN is hardly surprising since FPL data is inherently time-series data. Lindberg and Sonderberg (2020) showed that the data exhibits a recency bias - a player's recent performances is likely to predict their near-future performance. This is also congruent with what we see in reality. This invites the application of other machine learning techniques that are robust in handling time-series data.

2.3 Machine Learning Techniques for Time-Series Data

In what is described as the largest empirical study of Deep Neural Networks for Time Series Classification, Fawaz et al. (2019) concluded that Convolutional Neural Networks (CNNs) is the superior deep learning technique across both univariate and multivariate time-series problems.

CNNs are typically applied to images upon which their convolution layers process the dot product between a subset of that image (in its 2D numeric representation) and an array representing a 'filter', which is meant to highlight different characteristics of the input like edges or curves (LeCun et al. 1989).

CNNs have found success in textual, graphic and audio processing which, like time-series data, has a sequential or temporal dimension (Fawaz et al. 2019). The authors claim that the deeply flexible nature of CNN architectures permit robust feature extraction.

This stance is corroborated by Zhao et al. (2017) who showed that CNNs outperformed benchmarks and state-of-the-art time-series techniques of the time. The literature shows that CNNs can extract features that are relevant for specific time periods (Cui, W. Chen, and Y. Chen 2016) and are highly noise tolerant (Zhao et al. 2017).

Practically, CNNs have been shown to outperform LSTMs in time-series learning tasks ranging from supernova classification (Brunel et al. 2019) to forecasting COVID-19 cases (Nabi et al. 2021).

Hence, convolutional neural networks are promising candidates for time-series problems.

2.4 Neural Network Theory

A brief treatment of more elementary neural networks is provided before CNNs are discussed.

2.4.1 Perceptrons

A perceptron is essentially a mathematical function which computes whether the dot product of a vector of input values and a vector of weights (with the addition of some bias), will satisfy a predefined threshold/activation function.

Figure 2 shows an example of a perceptron with its typical activation function - a variant of the sign function, which outputs 1 if the input is greater than 0 and -1 otherwise. This enables the perceptron to classify some input into one of two categories - each possible output representing a distinct category. It should be observed that different activation functions would permit a different set of possible outputs. For example, the sign function which outputs 1 if the input is greater than 0, -1 if the input is less than 0 and 0 if the input is 0, permits three possible output categories. Similarly, the linear function $f = x$ permits an output of any real valued number.

A perceptron can be trained using the Perceptron Learning Algorithm, which iteratively updates the weight matrix in order to minimise the difference between the expected output and the actual output (Mitchell 1997).

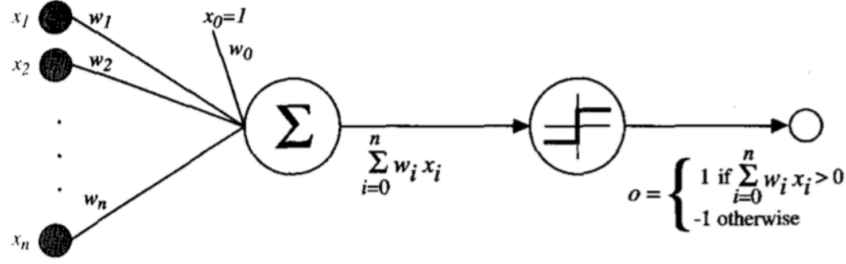


Figure 2: Model of a Perceptron (Nielsen 2015)

2.4.2 Multi-layer Perceptrons

As the name suggests, a multi-layer perceptron is a chain of many perceptrons, organised into layers. Typically, MLPs contain a single input layer, an arbitrary number of hidden layers and a single output layer. MLPs are feedforward and fully connected, which means that every perceptron in one layer is connected to every perceptron in the layer after it.

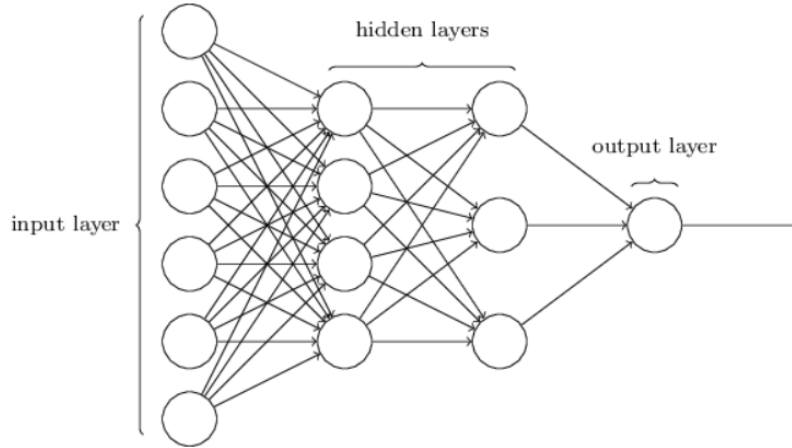


Figure 3: Model of an MLP (Mitchell 1997)

Output layers usually contain a threshold function that is differentiable to enable training via gradient descent. The ReLU function is currently pervasively used as the threshold function since it is simple and has been shown to consistently work well (Aleshin-Guendel 2017). Moreover, ReLU has been shown to be the average of cascaded sigmoids and to converge faster than the previously oft-used sigmoid activation function (Nair and Geoffrey E. Hinton 2010).

ReLU is defined as 0 for all negative inputs and the input value itself for all positive inputs; or more formally $ReLU(x) = \max(0, x)$. The network is trained via the backpropagation algorithm (Rumelhart, Geoffrey E Hinton, and Williams 1986).

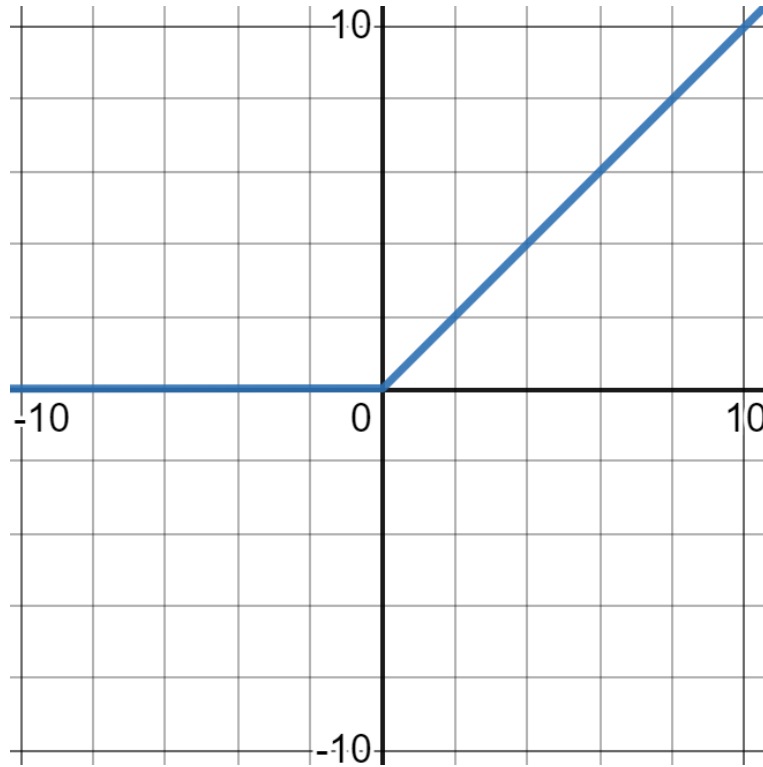


Figure 4: ReLU Function

2.4.3 Long Short-Term Memory Recurrent Neural Networks

As highlighted previously, LSTMs are gradient-based recurrent neural networks that can hold long-term information. They typically contain a cell state that stores important gradients over a long period of time, and is regulated by an input gate, output gate and forget gate (Hochreiter and Schmidhuber 1997). This architecture helps to ‘remember’ gradients over time that would otherwise have vanished (Olah 2015). This makes them well suited to handle time-series data (Amidi 2021).

2.4.4 Convolutional Neural Networks

In typical feedforward or recurrent neural networks, before data is fed into the network, it needs to be vectorised (i.e transformed into an $n \times 1$ column vector) since network weight matrices (with which dot products are computed) are typically also $n \times 1$ column vectors.

This limitation does not apply to CNNs due to their use of convolution operations.

Given some $n \times m$ input matrix F , and an $a \times b$ ‘filter’/weight matrix G , a convolution operation is defined as $F \cdot G$ (Hornik 1991).

This makes CNNs powerful for data that exists in some dimension(s) - like pixel data from an image (naturally 2 dimensional - which can be represented by an $n \times m$ matrix) or time-series data like the performance statistics of fantasy football players over a few gameweeks (which can be represented by an $n \times m$ matrix with n gameweeks and m distinct performance features).

CNNs therefore provide a unique opportunity to model input data as a time-series. Previous experiments (Gupta (2019) and Lindberg and Sonderberg (2020)) leveraged the “remembering” characteristic of LSTMs to account for the time-series nature of FPL data. With CNNs, input data could be represented as an actual time-series. More concretely, the performance of a player over a number of gameweeks could serve as a single input data instance and be represented as an $n \times m$ matrix with n gameweeks and m distinct performance features.

CNN architecture typically involves three layers - convolutional layers/kernels, pooling layers and fully connected layers.

The convolutional layers compute convolution operations on their inputs and are effectively the “weights” that are updated during training (via backpropagation).

The pooling layers apply some predefined subsampling operation to reduce/flatten the convolved matrix. The subsampling operation does not alter during training. Typically MaxPooling is applied, which extracts the highest value in the portion of the matrix being sampled and consequentially, the data attribute that is most impactful for the learning task.

Finally, a fully connected dense layer is followed by an output layer containing the activation functions required to output the necessary values/classifications.

Figure 5 shows an example of the architecture employed in the first CNN, LeNet.

(Hornik 1991)

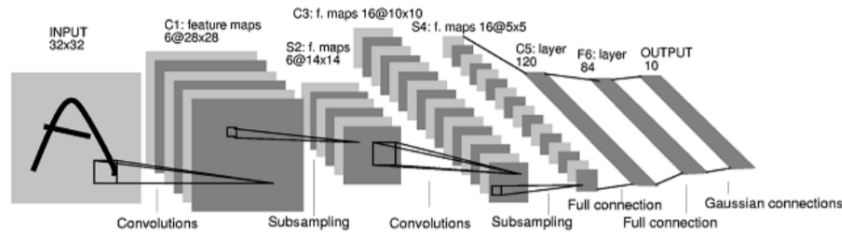


Figure 5: Example architecture of LeNet (Hornik 1991)

Lindberg and Sonderberg (2020) outlines an optimal network configuration and choice of hyperparameters for an MLP and an LSTM RNN. No literature exists that applies CNNs to the FPL prediction problem. As such, various combinations of network components and hyperparameters will need to be trailed in order to determine an optimal configuration for the CNN. These details will be further delineated in Chapter 3.

2.5 Training and Evaluation

Early stopping is a technique used to reduce a network’s chances of overfitting the training set, thereby increasing the network’s accuracy. Early Stopping advises that the network training halts after there is no significant reduction in the loss measured after some predefined number of training epochs (Morgan and Bourlard 1989).

Two distinct evaluation techniques emerge from the literature.

The first is to use the predicted scores to synthesise a fantasy team and compare the points accumulated by that team to fantasy managers globally (Stolyarov and Vasiliev 2017; Saifi 2018).

These evaluation techniques indicate how well the model performs in relation to other human players in the game.

The second is to use a loss function to calculate the error, individually or as a team, between the predictions and the actual results over some test dataset (Lutz 2015; Gupta 2019; Lindberg and Sonderberg 2020).

Lindberg and Sonderberg (2020) evaluated three loss functions - Mean Squared Error, Mean Absolute Error and Binary Cross-Entropy. The authors agreed with broad literature that Mean Squared Error (MSE) is the most appropriate loss function when approximating continuous variables using neural networks (commonly referred to as regression problems).

MSE is defined as the average of the square of the differences between the actual output and the expected output.

$$\frac{1}{n} \sum_{i=1}^D (x_i - y_i)^2$$

MSE is a popular loss function for two primary reasons. First, it is not difficult to compute or use during backpropagation. Second, the squares introduce exponentially greater penalisations for larger errors i.e. the network will be penalised much more heavily for a large deviation from the expected output, than for a smaller one - resulting in quicker corrections away from wildly incorrect weight values (Brownlee 2020).

2.6 Datasets and pre-processing

FPL related datasets are available online in a few different flavours. Gupta (2019) and Stolyarov and Vasiliev (2017) claim to have painstakingly scraped historical data off official FPL web pages. The Official FPL website also hosts a public API which serves fantasy data for the most recent season (*Fantasy Premier League API* 2021). Data since the 2016-17 FPL season is currently being maintained in an unofficial, community driven GitHub repository (*FPL GitHub Repository* 2021). Most studies hybridise the Official API for recent data and the GitHub repository for historical data (Saifi 2018; Lindberg and Sonderberg 2020; Bonello et al. 2019).

Saifi (2018), Gunjan (2013), and Bonello et al. (2019) all speak to discrepancies in data identifiers among different datasets and over different years within the same dataset i.e. different IDs denote the same player across datasets or years. The authors of the same sources also note that football players may share names – a case requiring disambiguation when analysing results.

Gunjan (2013) and Lindberg and Sonderberg (2020) trained and tested separate models for each fantasy football position (Goalkeeper, Defender, Midfielder and Forward). This reportedly increased the overall accuracy of predictions. The idea was first posited by Lutz (2015) to predict fantasy points for NFL players. The pervasiveness of this idea indicates that this is best practice when applying machine learning techniques to FPL datasets.

Gunjan (2013) reduced a dataset of 308 data features to just 34 and still yielded a prediction accuracy of 63.4%. Gunjan further showed how other attributes had negligible impact on the accuracy of the model.

3 Research Methodology

This section details the methodology employed to carry out the experiments. First, the pre-processing of raw data is described. This is followed by an explanation of how the input data was represented as a time-series. Then, the network architecture and hyperparameter combinations that were trialed during the experiments are discussed in tandem with the approach taken to train and test the networks. Thereafter, details about the programming implementation is provided. The section concludes with ethical considerations.

3.1 Data Pre-processing

The data for these experiments were sourced from a community driven data store of FPL footballer performance statistics hosted on GitHub (*FPL GitHub Repository* 2021). The data is stored in a csv file that contains statistics on footballers’ performance for every match that the footballer had played for a single season. For this experiment, the raw data for the 2020/2021 (latest completed) season was downloaded and imported into a python project.

A number of pre-processing transformations were applied to the raw data in order to make it easier to feed into the models. Pre-processing operations are detailed below and applied using the pandas and numpy python libraries.

First, data for each player was extracted from the raw data csv and written to individual csv files per player. Player csv files were then organised by the player’s position to facilitate easier training of different networks per position. At the end of this transform, there existed a csv file for each player, containing all performance data for all gameweeks that that player had played. Each row in the csv corresponded to a gameweek and each column, a certain detail or performance metric for that player for that gameweek. Table 2 shows a partial extract of a player csv for football player Aaron Connolly.

Gameweek	Name	xP	assists	...
1	Aaron Connolly	0.5	0	...
2	Aaron Connolly	4	0	...
3	Aaron Connolly	2.7	0	...
4	Aaron Connolly	2.7	0	...
5	Aaron Connolly	3	1	...

Table 2: Extract of a player csv

At this stage, it was discovered that the data was dirty in two ways.

Firstly, some player data was duplicated. For these cases, the dataset was traversed once more and duplicate rows were removed.

Secondly, if a player had not played in a certain gameweek, there was no row entry for that gameweek for that player. For these cases, the players were removed from the dataset. Problematically, this approach reduced the dataset size and therefore the number of data instances available for training. However, this solution was less harmful to the model’s accuracy as compared to the two possible alternatives, outlined below.

The first alternative was to insert zero-valued data for the missing gameweeks. However, zero-valued data means that a player did play but scored zero for the different performance attributes for that gameweek; which is untrue and could have perilous implications for what the networks learned.

The second alternative was to leave the data as is - with missing gameweeks. This is also perilous to the network’s accuracy since the input data was represented in a time-series which required predictable intervals.

Finally, data attributes unrelated to player performance (such as the player’s name and team) were removed. Table 3 lists the 23 performance-related data features that the input data contained after all pre-processing transformations.

Data Feature
xP
assists
bonus
bps
clean_sheets
creativity
element
goals_conceded
goals_scored
ict_index
influence
minutes
own-goals
penalties_missed
penalties_saved
red_cards
saves
team_a_score
team_h_score
threat
total_points
value
yellow_cards

Table 3: Data Features

3.2 Time-series Representation for Input Data

This section describes how the input data was structured as a time-series and the implications thereof. In essence, the goal was to feed the networks with many gameweeks worth of data features in a single input instance.

More concretely, recall that we have a dataset of performance metrics for footballers across all gameweeks for a particular season. A single instance of input data that is represented as a time-series, would be an aggregation of the 23 performance features available for each gameweek, across

n gameweeks. The number of gameweeks in a single input instance, n , is called the Input Window Size.

Formally, let D be a single input data instance. D is an $n \times m$ matrix with n gameweeks and m data features.

For this experiment, m will always be 23 (the number of distinct performance features yielded from the pre-processing stage).

In this experiment, different values for n (input window size) were trialed in order to determine an optimal configuration for the network. Consequently, the gameweek for which the network would need to make a prediction, would be different for each input window size (since the network predicts the result of the gameweek following the last gameweek in the data input). As a result, when the data was split into ‘features’ and ‘labels’ for training, the gameweek used for the set of labels was set to the gameweek following the last gameweek in the input (i.e. target gameweek number = input window size + 1).

3 different input window sizes were used - 3 gameweeks, 6 gameweeks and 9 gameweeks. As such, the networks were tasked with predicting the expected points return for each data instance in gameweeks 4, 7 and 10 respectively.

$$\begin{aligned} \text{Input Window Sizes } (n) &= \{3, 6, 9\} \\ \text{Number of data features } (m) &= \{23\} \end{aligned}$$

An example matrix with 2 input data instances (D), each containing multiple data features for 3 gameweeks.

$$\begin{bmatrix} \begin{bmatrix} 0 & 5 & 0.2 & \dots & 6 & 0.9 & 3 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 & 0 & \dots & 2 & 0 & 1 \end{bmatrix} \\ \begin{bmatrix} 1 & 2 & 0 & \dots & 3 & 0.5 & 6 \end{bmatrix} \end{bmatrix}$$

This time-series representation for the input data was used for all networks in this research.

Notably for the CNN implementation, the time-series representation works well when using a 1D convolutional layer.

1D convolutional layers employ $n \times m$ kernels where m is set to the number of data features, in this case 23. That is to say that all kernels have a “horizontal” size that is the same as the number of data features, since they only need to perform convolutions “vertically” from the beginning of the time-series (first gameweek) to the end of the time-series (n^{th} gameweek). This is in contrast to more typical 2D convolution operations wherein the kernel slides horizontally over the matrix as well.

Figure 6 illustrates a CNN architecture leveraging 1D convolutions for time-series data.

3.3 Network Architecture, Hyperparameters, Training and Evaluation

4 iterations of each network were trained and evaluated separately; one for each football position (goalkeepers, defenders, midfielders, forwards). All networks used ReLU activation functions and contained an output layer with a single Dense neuron.

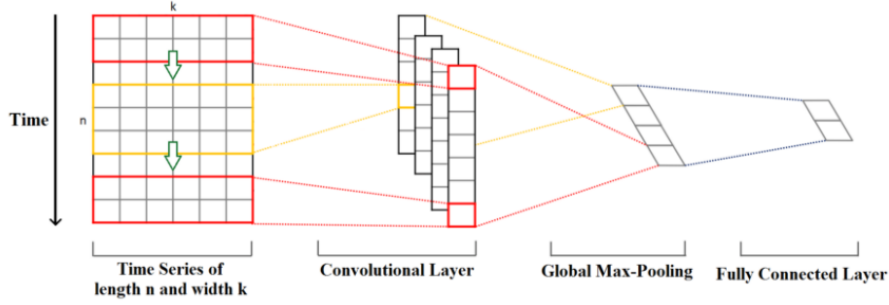


Figure 6: Overview of end-to-end operations in a CNN leveraging 1D convolutions for time-series data (Granat 2019)

3.3.1 CNN

The CNN was implemented with a single 1D convolutional layer followed by a single MaxPooling layer and a single fully connected Dense layer before funneling into a single neuron in the final Dense output layer. The fully connected Dense layer contained as many neurons per football position as Lindberg and Sonderberg (2020) had found to yield the best performance, noted in Table 4

The model was trained independently for every football position, using every combination of the following parameters, in order to find the best configuration per footballer position.

$$\begin{aligned} \text{Input Window Size (number of gameweeks)} &= \{3, 6, 9\} \\ \text{Kernel size} &= \{1, \dots, \text{Input Window Size} - 1\} \\ \text{Number of filters} &= \{32, 64, 128, 256\} \end{aligned}$$

3.3.2 Networks used for Comparison

Lindberg and Sonderberg (2020) trialed varying combinations of hyperparameters with different network architectures for their MLP and LSTM RNN models. This experiment has adopted the same architecture and hyperparameters that the authors had reported as having the best success with. Since these networks were used for comparison only (i.e. an optimal configuration did not need to be discovered), they were trained and tested using input data with a window size of 3 only.

MLP

Position	Hidden Layers	Neurons in each hidden layer
Goalkeeper	6	64
Defenders	1	128
Midfielders	1	64
Forwards	1	64

Table 4: Optimal MLP Architecture (Lindberg and Sonderberg 2020)

LSTM RNN

Position	Hidden LSTM-Layers	Neurons in each hidden LSTM-layer	Neurons in last hidden layer
Goalkeeper	2	256	64
Defenders	1	64	64
Midfielders	3	128	128
Forwards	2	64	128

Table 5: Optimal LSTM RNN Architecture (Lindberg and Sonderberg 2020)

3.3.3 Training

All networks were trained for 100 epochs. Early stopping was applied should the network have been found to be non-improving for 20 epochs. Each model was trained independently per footballer position. The dataset was split into training and test sets using the ratio 2:1.

3.3.4 Evaluation and Analysis

The Mean Squared Error (MSE) was used as the loss function during training. The MSE was also collected and used to compare the results of each network against each other and previous findings in literature. Since the MSE is a numerical value of statistical significance, the data collection is considered to be quantitative in nature.

Python’s `matplotlib` library was used to generate graphs of the results. The network’s output logs from training and evaluation were saved as text files.

3.4 Programmatic Implementation

The neural networks were implemented programmatically using Python 3.9 and executed on an Intel Core i7 x64 CPU with approximately 30GB of usable RAM.

From a programming perspective, the networks were implemented using Tensorflow’s `Dense`, `Conv1D` and `LSTM` (*TensorFlow* 2021). `Dense`, `Conv1D` and `LSTM` are `Keras` layers used to implement a layer of fully connected perceptrons, 1-D convolutional neural networks and long short-term memory recurrent neural networks, respectively. `NumPy` and `Pandas` were leveraged as utility libraries as well.

The GitHub repository for all code used for this experiment, including the networks and data transformations, can be found at the following reference (Ramdas 2021).

3.5 Ethical Considerations

This research was conducted under class ethical clearance for the topic ‘Applications of Machine Learning’. All data used in this research is publicly available and free to use. All software tools and libraries used in this research are open-source and free to use.

4 Data Analysis and Results

4.1 Training the CNN

In order to determine an optimal configuration, the CNN was trained using every combination of the following parameters:

$$\begin{aligned}\text{Input Window Size (number of gameweeks)} &= \{3, 6, 9\} \\ \text{Kernel size} &= \{1, \dots, \text{Input Window Size} - 1\} \\ \text{Number of filters} &= \{32, 64, 128, 256\}\end{aligned}$$

Figures 7, 8, 9 and 10 display the loss (mean squared error) measured over 100 epochs of training for 4 of the 264 combinations trailed during training. These 4 combinations were specifically chosen as they constitute the set of parameter combinations that performed the best for each position.

It is observed that none of the four optimal configurations ceded to early stopping during training. Although training after epoch 40 resulted in very minimal improvement, all configurations did see improvement all the way to the 100th epoch.

4.2 Optimal Configuration for CNN

After training and testing, the configurations in Table 6 were found to yield the least mean squared error on the test set (indicating the best performance). The results for all parameter combinations are charted in Figures 11 - 22.

Position	Time Interval (in number of gameweeks)	Number of Filters	Kernel Size	MSE
Goalkeepers	6	128	4	0.808
Defenders	9	128	8	1.375
Midfielders	6	128	2	1.294
Forwards	3	32	1	1.912

Table 6: Optimal CNN configuration per position

There is no obvious observable relationship between the MSE and any of the dependent variables - Input Window Size, Number of Filters, Kernel Size.

Of interest is the MSE resultant from using an input window size of 6 with 128 filters for goalkeepers (see Figure 12). This configuration saw both the highest and lowest MSE values recorded across all configurations. Specifically, a kernel size of 3 recorded an MSE of 220.843; while a kernel size of just one more, 4, recorded an MSE of 0.808.

These results conclude the first objective of this research - to successfully apply convolutional neural networks to the fantasy premier league prediction problem.

4.3 Comparison with other networks

An MLP and LSTM RNN were also constructed and configured with hyperparameters previously found to yield the lowest error (Lindberg and Sonderberg 2020).

Table 7 catalogues a comparison of the MSE reported on the MLP and LSTM RNN for both, this experiment ("Exp MLP" and "Exp LSTM RNN"), and previous best performance recorded in literature ("Lit MLP" and "Lit LSTM RNN") (Lindberg and Sonderberg 2020). The performance of the CNN is also tabulated for comparison.

These results show that the CNN performed better (lowest MSE) than the MLP and LSTM RNN in both this experiment and in literature, across all positions.

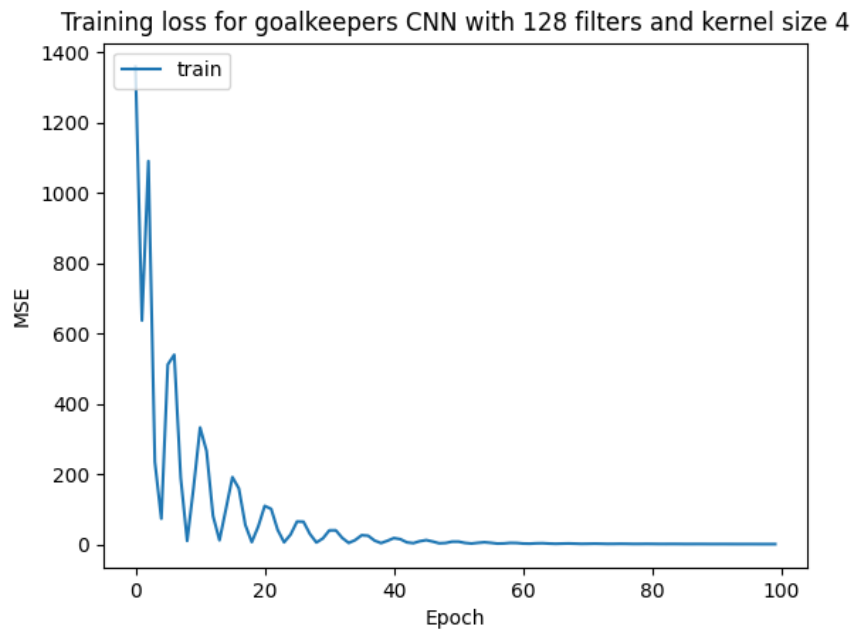


Figure 7: MSE over 100 epochs for goalkeepers

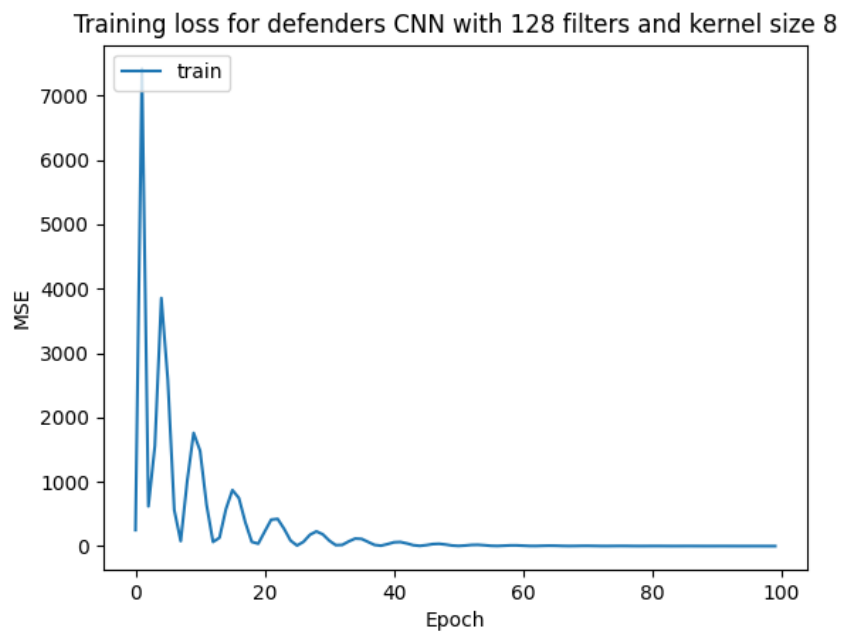


Figure 8: MSE over 100 epochs for defenders

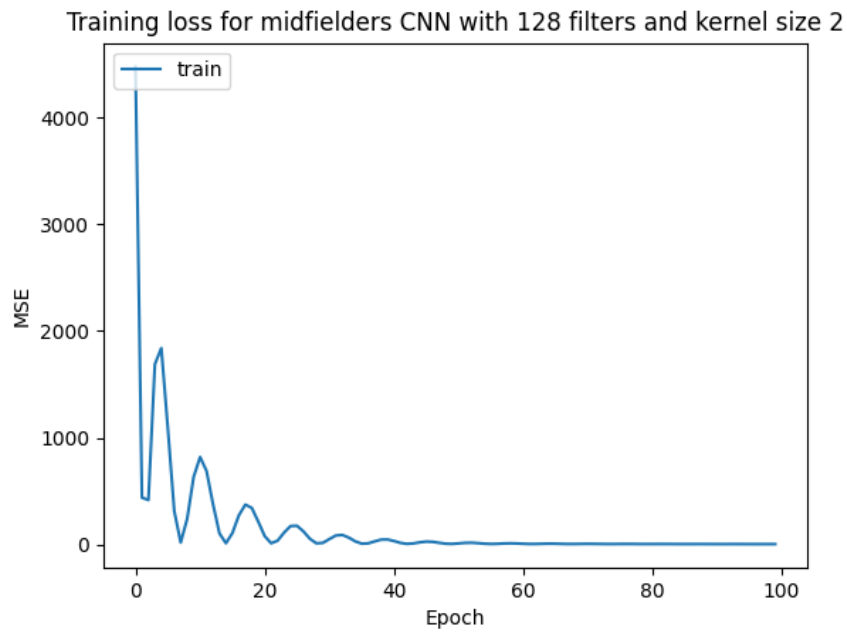


Figure 9: MSE over 100 epochs for midfielders

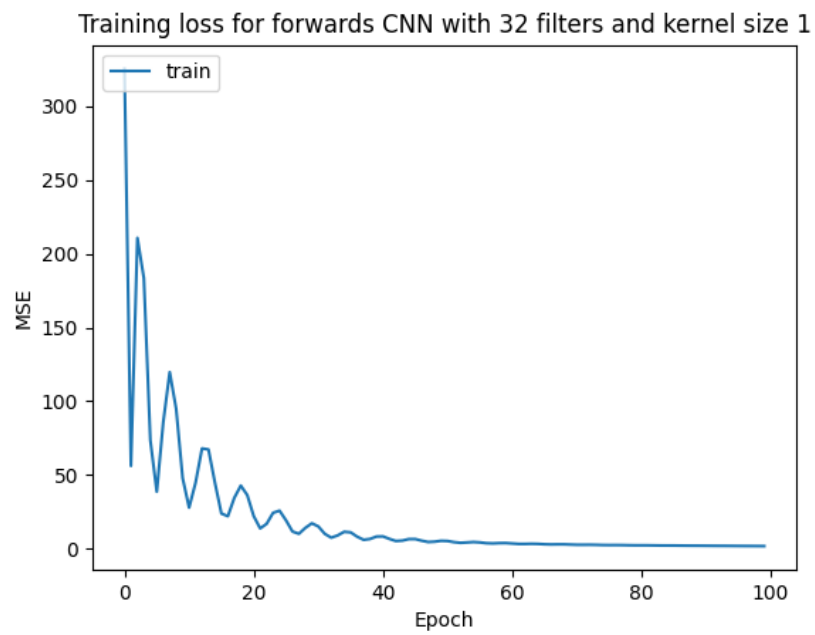


Figure 10: MSE over 100 epochs for forwards

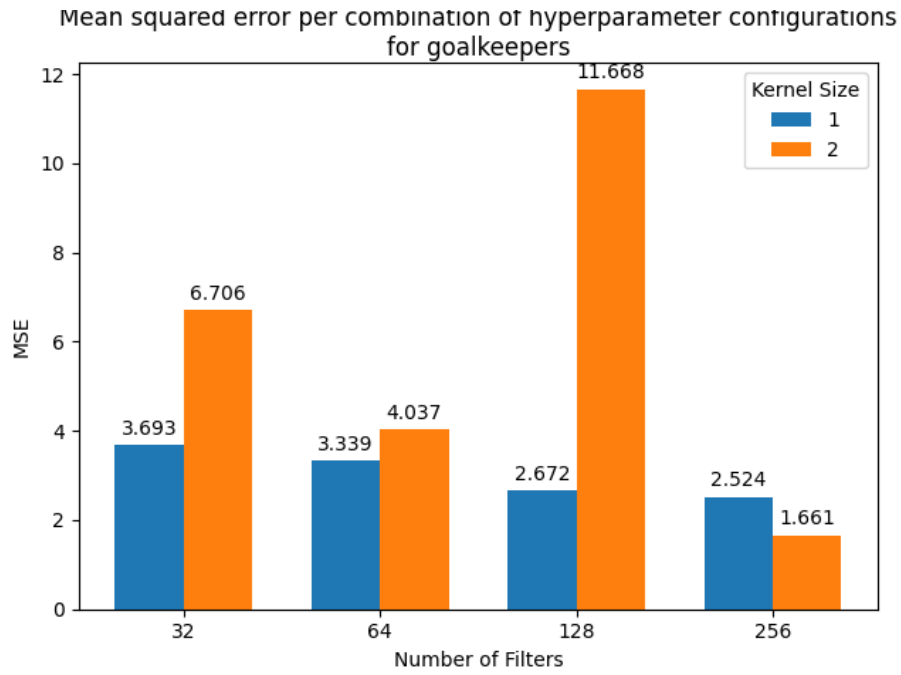


Figure 11: MSE per hyperparameter combination for goalkeepers with a time interval of 3 gameweeks

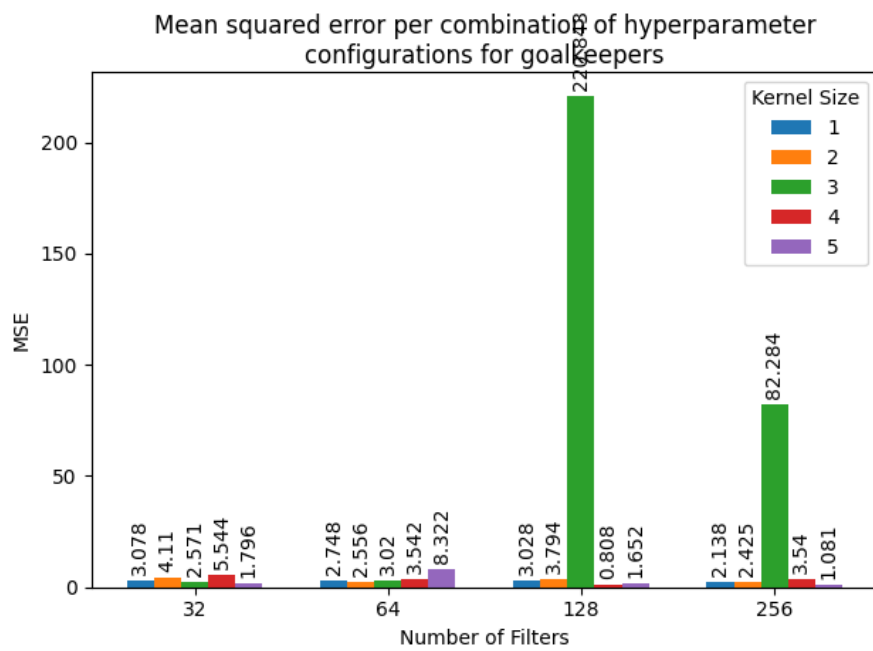


Figure 12: MSE per hyperparameter combination for goalkeepers with a time interval of 6 gameweeks

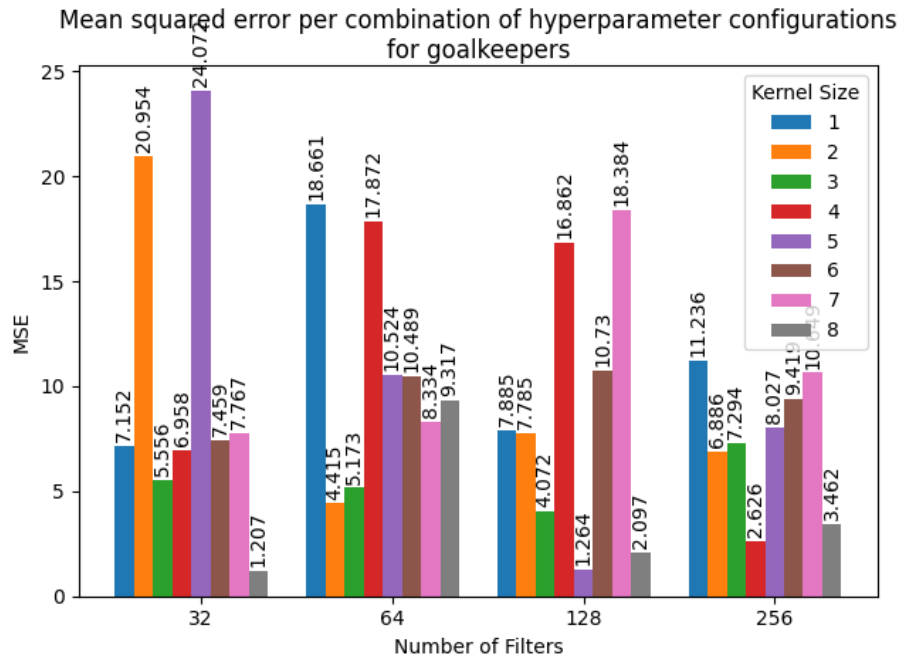


Figure 13: MSE per hyperparameter combination for goalkeepers with a time interval of 9 gameweeks

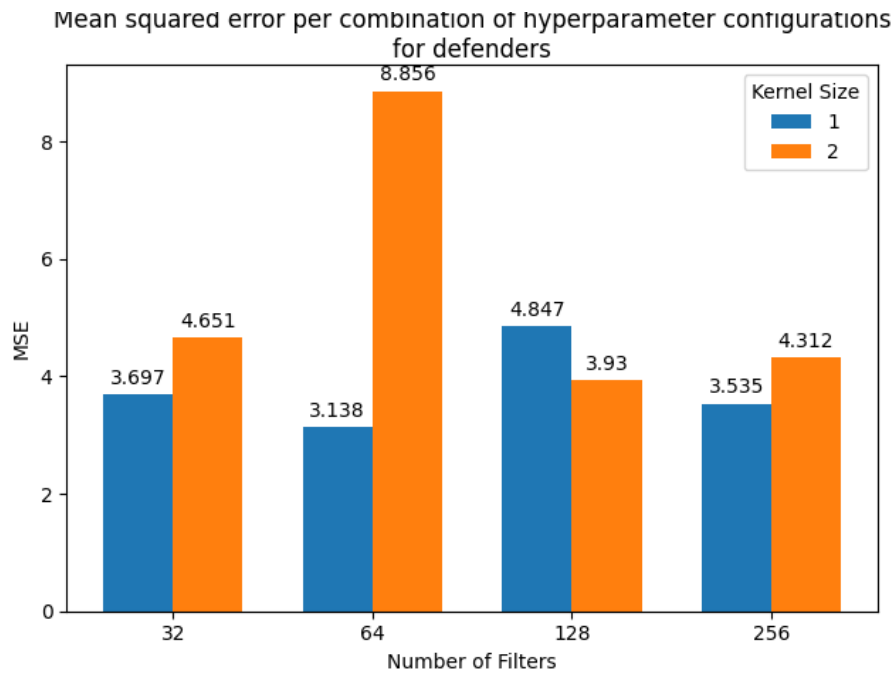


Figure 14: MSE per hyperparameter combination for defenders with a time interval of 3 gameweeks

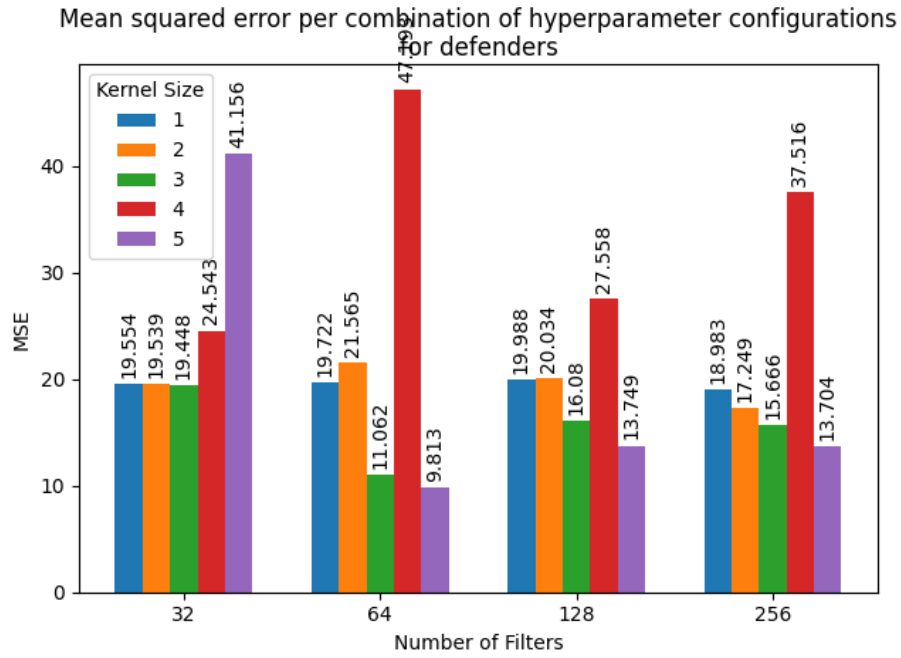


Figure 15: MSE per hyperparameter combination for defenders with a time interval of 6 gameweeks

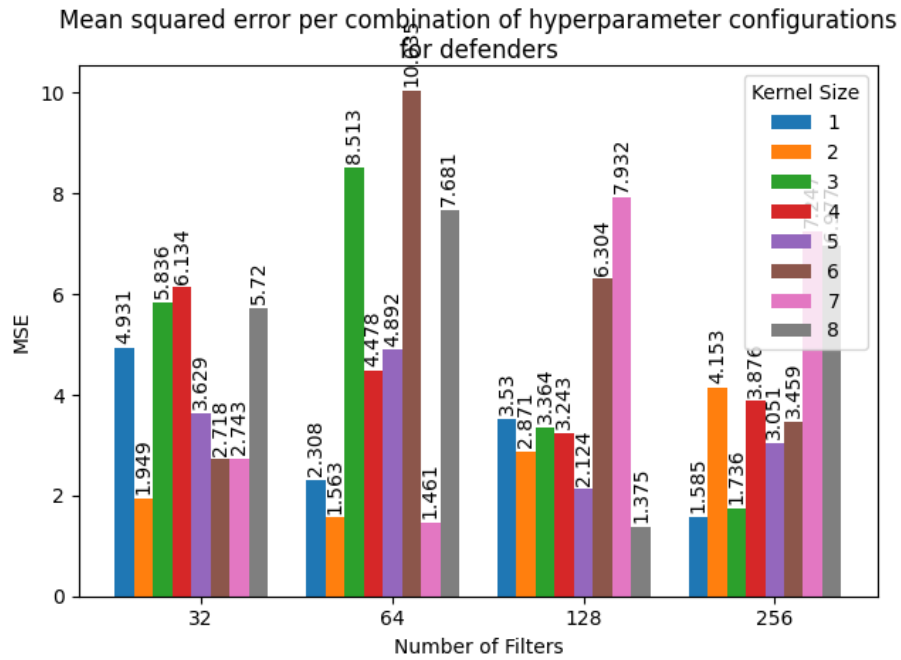


Figure 16: MSE per hyperparameter combination for defenders with a time interval of 9 gameweeks

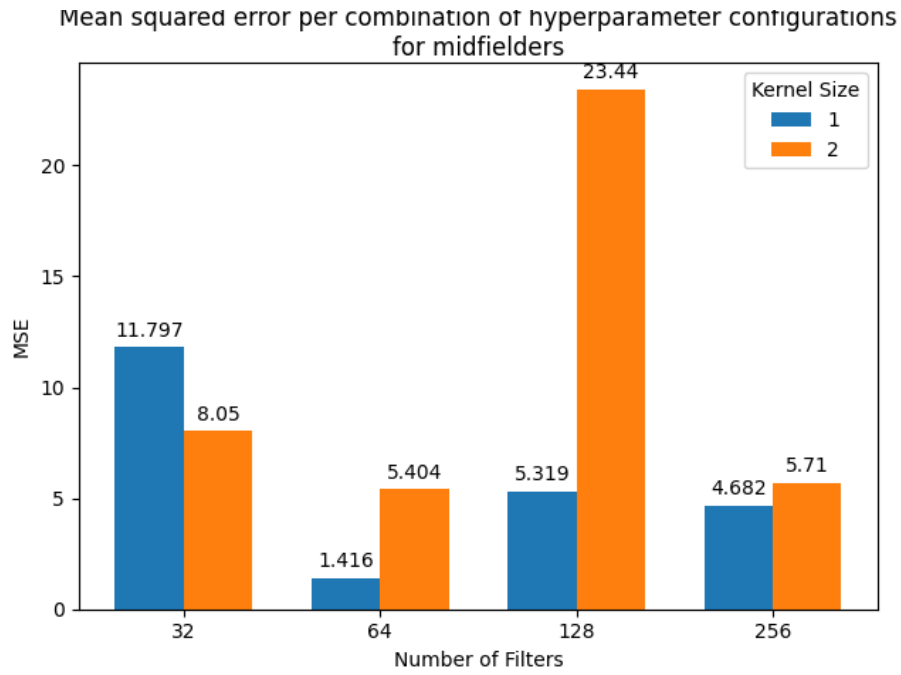


Figure 17: MSE per hyperparameter combination for midfielders with a time interval of 3 gameweeks

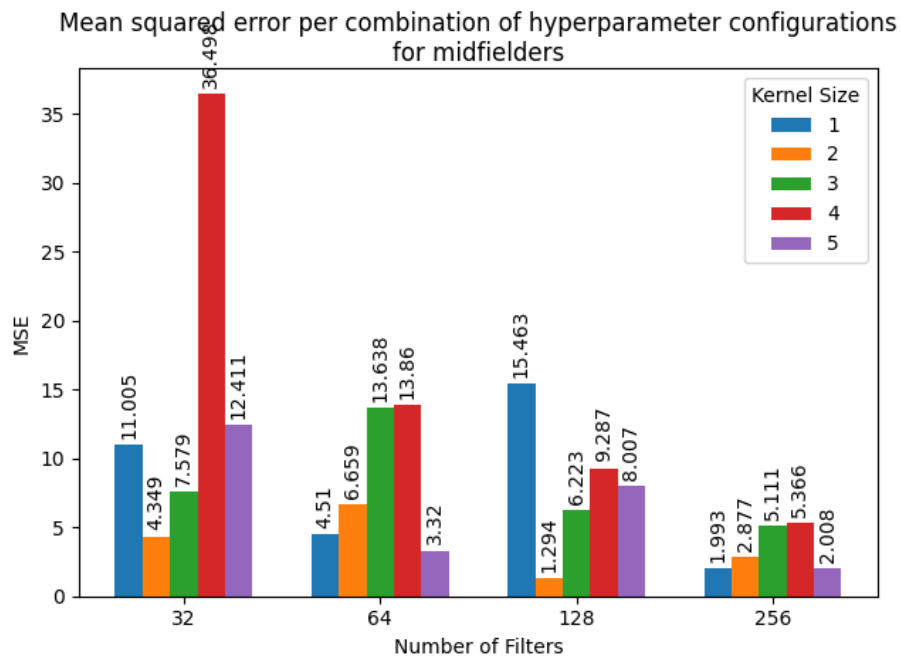


Figure 18: MSE per hyperparameter combination for midfielders with a time interval of 6 gameweeks

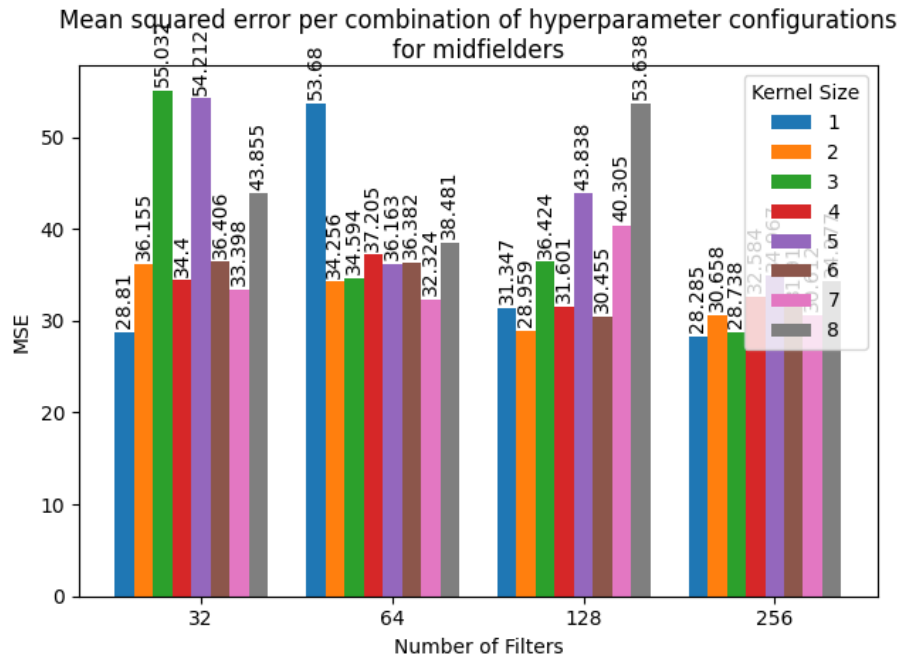


Figure 19: MSE per hyperparameter combination for midfielders with a time interval of 9 gameweeks

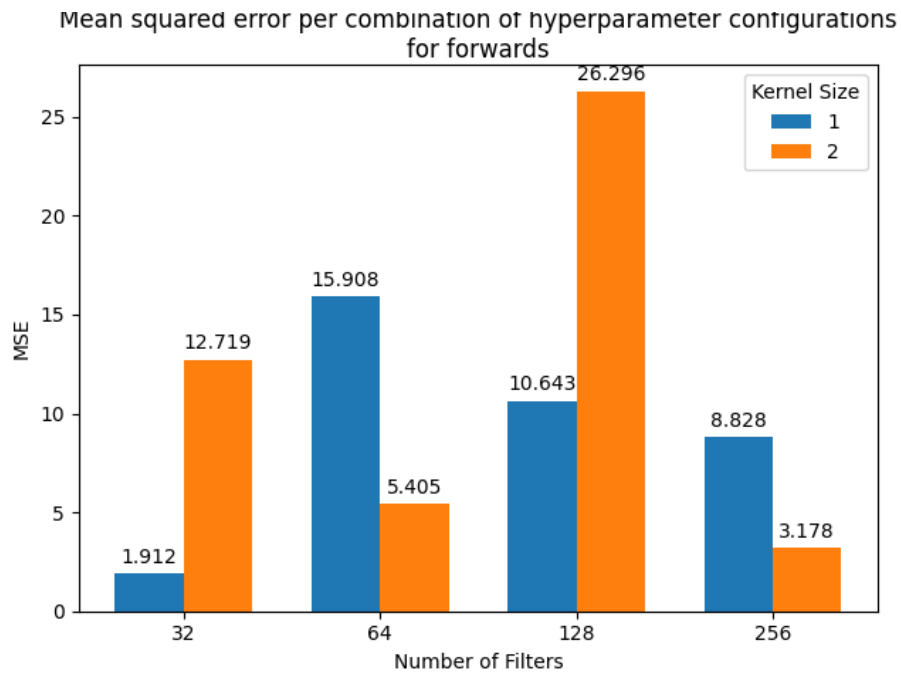


Figure 20: MSE per hyperparameter combination for forwards with a time interval of 3 gameweeks

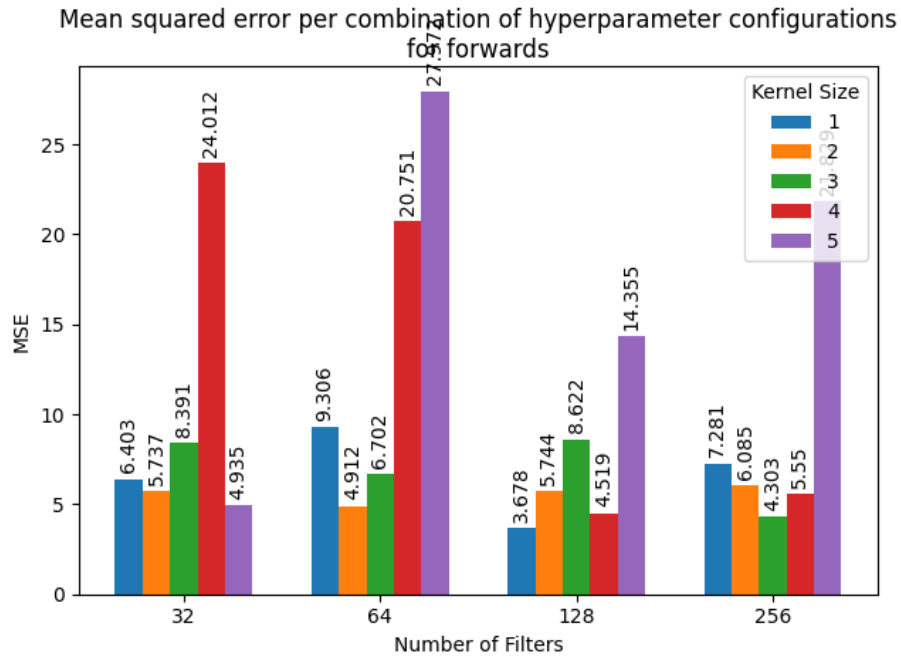


Figure 21: MSE per hyperparameter combination for forwards with a time interval of 6 gameweeks

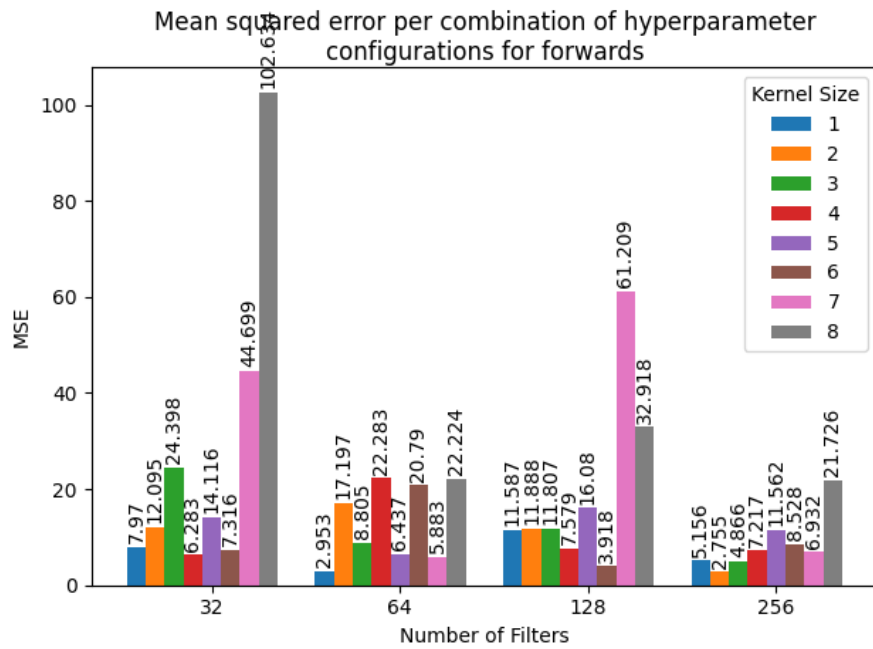


Figure 22: MSE per hyperparameter combination for forwards with a time interval of 9 gameweeks

Position	Lit MLP	Exp MLP	Lit LSTM RNN	Exp LSTM RNN	CNN
Goalkeepers	9.65	5.89	8.59	4.75	0.808
Defenders	9.20	4.00	8.31	2.84	1.38
Midfielders	8.05	1.89	6.88	2.11	1.29
Forwards	9.65	2.89	9.65	4.79	1.91
Average	9.03	3.67	8.36	3.62	1.35

Table 7: Comparison of the MSE for different machine learning techniques applied to the FPL prediction problem (the lower, the better)

Additionally, the MLP and LSTM RNN constructed for this experiment outperformed their counterparts in literature. This indicates that modelling the FPL prediction problem as a time-series problem - with input data spanning a number of gameweeks as opposed to the traditional single gameweek - results in better model performance.

Crucially, in the literature, the LSTM RNN was reported to have outperformed the MLP across all positions. In this experiment, the LSTM RNN had outperformed the MLP in 3 of the 4 positions. Interestingly, this experiment’s MLP outperformed the LSTM RNN of literature across all positions and had an average MSE lower than literature’s best by 4.69.

Once again, these results reveal the power of using time-series representations to model input data.

These results conclude the second objective of this research - to compare the performance of convolutional neural networks on the fantasy premier league prediction problem to other machine learning techniques.

5 Conclusion

In this report, convolutional neural networks were studied and benchmarked against long short-term memory recurrent neural networks and multi-layer perceptrons, in predicting the performance of fantasy football players in the fantasy premier league.

The network successfully computed predictions with a low margin of error, and an optimal parameter configuration - including the input window size, number of filters and kernel size - was established. This contribution satisfied the objective of successfully applying a CNN to the FPL prediction problem.

The network outperformed previous best-performing techniques in the domain - the LSTM RNN and MLP - yielding a much lower mean squared error across all four football positions. This result satisfied the objective of comparing the CNN to other machine learning techniques.

Chapter 4 outlined the performance of a CNN optimally configured, in predicting players’ points returns in FPL, thereby answering the original research question - “How do Convolutional Neural Networks perform in the prediction of player points in the Fantasy Premier League?”

Thus, this report contributes to the existing body of knowledge on the application of machine learning techniques in the fantasy football domain in three ways. Firstly, it has proven that CNNs can successfully be applied to predictor problems upon the fantasy premier league dataset. Secondly, it has presented an optimal configuration for those CNN implementations. Thirdly, it has shown how CNNs can outperform other machine learning techniques on the FPL prediction problem.

This research was most limited by the available input data. The input data required much pre-processing and was limited in size. A more programmatic-friendly data store (like a normalised database) of FPL data would aid future research in this domain. Additionally, future work could make use of more than one season of data.

In order to further this research, more complex CNN architectures should be considered. Multiple convolutional and subsampling layers could be trialed, along with additional training aids such as regularization or dropout. Additional hyperparameter configurations can be experimented with, including the stride length or dilation rate.

More rigorous data normalisation techniques can also be evaluated.

Future work could also focus on applying less prescription over the input window size. Should the CNN be designed such that the input window size can be variable, players that do not play matches regularly could be included in the input dataset.

References

- Aleshin-Guendel, Serge (2017). “Examining the Structure of Convolutional Neural Networks”. In: Amidi, Shervine (Jan. 2021). *Recurrent Neural Networks*. URL: <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks>.
- Bonello, Nicholas, Joeran Beel, Seamus Lawless, and Jeremy Debattista (2019). “Multi-stream Data Analytics for Enhanced Performance Prediction in Fantasy Football”. In: *arXiv preprint arXiv:1912.07441*.
- Bonomo, Flavia, Guillermo Duran, and Javier Marenco (Dec. 2013). “Mathematical programming as a tool for virtual soccer coaches: A case study of a fantasy sport game”. In: *International Transactions in Operational Research* 21. DOI: 10.1111/itor.12068.
- Brownlee, Jason (Aug. 2020). *How to Choose Loss Functions When Training Deep Learning Neural Networks*. URL: <https://machinelearningmastery.com/how-to-choose-loss-functions-when-training-deep-learning-neural-networks/>.
- Brunel, Anthony, Johanna Pasquet, Jérôme Pasquet, Nancy Rodriguez, Frédéric Comby, Dominique Fouchez, and Marc Chaumont (Jan. 2019). “A CNN adapted to time series for the classification of Supernovae”. In:
- Chen, Tianqi and Carlos Guestrin (2016). “Xgboost: A scalable tree boosting system”. In: *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pp. 785–794.
- Comrie, Andrew (1997). “Comparing neural networks and regression models for ozone forecasting”. In: *Journal of the Air & Waste Management Association* 47.6, pp. 653–663.
- Cortes, Corinna and Vladimir Vapnik (1995). “Support-vector networks”. In: *Machine learning* 20.3, pp. 273–297.
- Cui, Zhicheng, Wenlin Chen, and Yixin Chen (2016). *Multi-Scale Convolutional Neural Networks for Time Series Classification*. eprint: [arXiv:1603.06995](https://arxiv.org/abs/1603.06995).
- Esling, Philippe and Carlos Agon (Dec. 2012). “Time-Series Data Mining”. In: *ACM Comput. Surv.* 45.1. ISSN: 0360-0300. DOI: 10.1145/2379776.2379788. URL: <https://doi.org/10.1145/2379776.2379788>.
- Fantasy Premier League* (2021). URL: <https://fantasy.premierleague.com/>.
- Fantasy Premier League API* (2021). URL: <https://fantasy.premierleague.com/api/bootstrap-static/>.
- Fantasy Premier League Rules* (2021). URL: <https://fantasy.premierleague.com/help/rules>.
- Fawaz, Hassan Ismail, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller (Mar. 2019). “Deep learning for time series classification: a review”. In: *Data Mining and Knowledge Discovery* 33.4, pp. 917–963. DOI: 10.1007/s10618-019-00619-1. URL: <https://doi.org/10.1007/s10618-019-00619-1>.
- FPL GitHub Repository* (2021). URL: <https://github.com/vaastav/Fantasy-Premier-League>.
- FPL Statistics for Heung-Min Son* (2021). URL: https://github.com/vaastav/Fantasy-Premier-League/blob/master/data/2020-21/players/Heung-Min_Son_390/history.csv.

- Friedman, Jerome (2001). “Greedy function approximation: a gradient boosting machine”. In: *Annals of statistics*, pp. 1189–1232.
- Gangal, Anurag, Abhishek Talnikar, Aneesh Dalvi, Vidya Zope, and Aadesh Kulkarni (Dec. 2015). “Analysis and Prediction of Football Statistics using Data Mining Techniques”. In: *International Journal of Computer Applications* 132, pp. 8–11. DOI: 10.5120/ijca2015907263.
- Gers, Felix (May 2001). “Long Short-Term Memory in Recurrent Neural Networks”. In: DOI: 10.5075/epfl-thesis-2366.
- Granat, Margarita (Oct. 2019). *How to Use Convolutional Neural Networks for Time Series Classification*. URL: <https://towardsdatascience.com/how-to-use-convolutional-neural-networks-for-time-series-classification-56b1b0a07a57>.
- Gunjan, Kumar (2013). “Machine learning for soccer analytics”. In: *University of Leuven*.
- Gupta, Akhil (Sept. 2019). *Time Series Modeling for Dream Team in Fantasy Premier League*.
- Hamadani, Babak (2005). *Predicting the outcome of NFL games using machine learning*.
- Hochreiter, Sepp and Jürgen Schmidhuber (Nov. 1997). “Long Short-Term Memory”. In: *Neural Computation* 9.8, pp. 1735–1780. ISSN: 0899-7667. DOI: 10.1162/neco.1997.9.8.1735. eprint: <https://direct.mit.edu/neco/article-pdf/9/8/1735/813796/neco.1997.9.8.1735.pdf>. URL: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Hornik, Kurt (1991). “Approximation capabilities of multilayer feedforward networks”. In: *Neural Networks* 4.2, pp. 251–257. ISSN: 0893-6080. DOI: [https://doi.org/10.1016/0893-6080\(91\)90009-T](https://doi.org/10.1016/0893-6080(91)90009-T). URL: <https://www.sciencedirect.com/science/article/pii/089360809190009T>.
- Hubáček, Ondřej, Gustav Šourek, and Filip Železný (Jan. 2019). “Learning to predict soccer results from relational data with gradient boosted trees”. en. In: *Machine Learning* 108.1, pp. 29–47. ISSN: 0885-6125, 1573-0565. DOI: 10.1007/s10994-018-5704-6. URL: <http://link.springer.com/10.1007/s10994-018-5704-6> (visited on 06/18/2021).
- LeCun, Yann, Bernhard Boser, John Denker, Donnie Henderson, Richard Howard, Wayne Hubbard, and Lawrence Jackel (1989). “Handwritten digit recognition with a back-propagation network”. In: *Advances in neural information processing systems* 2.
- Lindberg, Adrian and David Sonderberg (2020). “Comparison of Machine Learning Approaches Applied to Predicting Football Players Performance”. English. MA thesis. Gothenburg: Chalmers University of Technology and University of Gothenburg.
- Lutz, Roman (2015). “Fantasy football prediction”. In: *arXiv preprint arXiv:1505.06918*.
- Marzban, C. and G. Stumpf (1996). “A Neural Network for Tornado Prediction Based on Doppler Radar-Derived Attributes”. In: *Journal of Applied Meteorology* 35, pp. 617–626.
- Matthews, Tim, Sarvapali Ramchurn, and Georgios Chalkiadakis (July 2012). “Competing with humans at fantasy football: team formation in large partially-observable domains”. In:
- Mitchell, Tom (1997). *Machine learning*. McGraw-hill New York.
- Morgan, Nelson and Hervé Bourlard (1989). “Generalization and parameter estimation in feedforward nets: Some experiments”. In: *Advances in neural information processing systems* 2, pp. 630–637.

- Nabi, Khondoker Nazmoon, Md Toki Tahmid, Abdur Rafi, Muhammad Ehsanul Kader, and Md. Asif Haider (2021). “Forecasting COVID-19 cases: A comparative analysis between recurrent and convolutional neural networks”. In: *Results in Physics* 24, p. 104137. ISSN: 2211-3797. DOI: <https://doi.org/10.1016/j.rinp.2021.104137>. URL: <https://www.sciencedirect.com/science/article/pii/S2211379721002904>.
- Nair, Vinod and Geoffrey E. Hinton (2010). “Rectified Linear Units Improve Restricted Boltzmann Machines”. In: *ICML*.
- Nielsen, Michael A. (Jan. 2015). *Neural Networks and Deep Learning*. URL: <http://neuralnetworksanddeeplearning.com/chap1.html>.
- Olah, Christopher (2015). *Understanding lstm networks*. URL: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>.
- Pardee, M (1999). “An Artificial Neural Network Approach to College Football Prediction and Ranking”. In:
- Raghunandh, GS (2018). *Building an FPL Captain Classifier*. URL: <https://medium.com/datacomics/building-an-fpl-captain-classifier-cf4ee343ebcc>.
- Ramdas, Delano Duran (2021). *fpl-predictor-neural-nets*. URL: <https://github.com/DelanoDZR/fpl-predictor-neural-nets>.
- Rudy, Kevin (2011). *Using regression analysis to predict fantasy football scores*. URL: <https://blog.minitab.com/en/the-statistics-game/using-regression-analysis-to-predict-fantasy-football-scores>.
- Rumelhart, David E, Geoffrey E Hinton, and Ronald J Williams (1986). “Learning representations by back-propagating errors”. In: *nature* 323.6088, pp. 533–536.
- Saifi, Murtaza (2018). “Implementation of Machine Learning Techniques to Predict Player Performance using Underlying Statistics”. PhD thesis. Dublin, National College of Ireland.
- Sierra, Adrian, James Fosco, and Carlos Fierro (2020). “Football futures”. In: *URL http://cs229.stanford.edu/proj2011/SierraFoscoFierro-FootballFutures.pdf*.
- Stolyarov, Arseniy and Gleb Vasiliev (2017). “Predict To Succeed: Optimal Sequential Fantasy Football Squad Formation Using Machine Learning Tools”. In:
- TensorFlow* (2021). URL: <https://www.tensorflow.org/>.
- Thakkar, Ankit and Kinjal Chaudhari (2021). “A comprehensive survey on deep neural networks for stock market: The need, challenges, and future directions”. In: *Expert Systems with Applications* 177, p. 114800. ISSN: 0957-4174. DOI: <https://doi.org/10.1016/j.eswa.2021.114800>. URL: <https://www.sciencedirect.com/science/article/pii/S0957417421002414>.
- Timmaraaju, Aditya Srinivas, Aditya Palnitkar, and Vikesh Khanna (2013). “Game ON! Predicting English Premier League Match Outcomes”. In:
- Ulmer, Ben and Matthew Fernandez (2014). “Predicting Soccer Match Results in the English Premier League”. In:
- Zhao, Bendong, Huanzhang Lu, Shangfeng Chen, Junliang Liu, and Dongya Wu (Feb. 2017). “Convolutional neural networks for time series classification”. In: *Journal of Systems Engineering and Electronics* 28, pp. 162–169. DOI: 10.21629/JSEE.2017.01.18.